

Cloud-Native Microservices Platform on Amazon EKS

A plain-English explanation of the project, its components, its workflows and its architecture

The résumé line this document explains

“Led the full lifecycle of multiple services from architecture and design to development, testing, and maintenance, shipping Docker images on Kubernetes (EKS) with workload identities and ingress gateways, CI/CD in AWS CodePipeline, and 80%+ JUnit 5 / Mockito coverage.”

Contents

Section	What it covers
1. Overall Summary in Simple Terms	The whole project in a few paragraphs, a restaurant analogy, and the résumé line decoded phrase by phrase.
2. The Big Picture	Figure 1 — one diagram showing the journey from a developer's keyboard to a real user.
3. Every Component Explained, End to End	The microservices, lifecycle ownership, Docker, ECR, Kubernetes, EKS, workload identity, ingress gateways, CodePipeline, JUnit 5 / Mockito testing, and observability.
4. The Workflows, End to End	Four step-by-step walkthroughs: code to production, one user request, how a pod gets AWS access, and what happens when things break.
5. The Architecture Diagram, Explained Layer by Layer	Figure 4 and a line-by-line explanation of every box, boundary and arrow in it.
6. Glossary	Every technical term used in this document, in one sentence each.
7. How to Explain This Out Loud	A 30-second version, a two-minute version, and the questions to expect.

1. Overall Summary in Simple Terms

In one sentence: I built several small backend applications that together power one product — I designed them, wrote them, tested them, packed each one into a standard container, and set up the automation that ships them, on its own, onto a cloud platform that keeps them running day and night.

Instead of one huge program that does everything, the system is split into several small programs. Each one has a single job — one handles orders, one handles payments, one handles user accounts. Each is packed into a sealed box (a Docker image) so it behaves identically on a laptop, in testing and in production. Those boxes run on a cluster managed by Amazon (EKS), which starts them, restarts them when they crash, and adds more copies when the site gets busy. All traffic from the outside world enters through a single guarded front door (the ingress gateway). Each service carries its own ID badge (a workload identity), so it can reach the database or file storage it needs without any password ever being written into the code. And every time a developer merges a change, an automated assembly line (AWS CodePipeline) rebuilds it, runs hundreds of automated tests, and — only if at least 80% of the code is covered by passing tests — releases it.

1.1 The Same Thing, Explained as a Restaurant

If the technology words are unfamiliar, this analogy maps every piece onto something ordinary. Every later section refers back to it.

The technical piece	Its restaurant equivalent	Why that matters
Microservices	Specialist kitchen stations: grill, salads, desserts. Each does one job.	You can change the dessert recipe without shutting the grill.
Docker image	A sealed prep kit: recipe, ingredients and tools in one box.	Open it in any kitchen and you get exactly the same dish.
Kubernetes / EKS	The head chef running the floor.	Replaces a cook who walks out, adds cooks on a busy night, decides who stands where.
Ingress gateway	The host at the front door.	Reads what the customer wants and points them to the right station. Nobody wanders into the kitchen.
Workload identity	A personal keycard for each station.	Opens only the cupboards that station needs — no master key taped under the counter.
CI/CD (CodePipeline)	The conveyor from the test kitchen to every branch.	A new recipe travels through tasting and safety checks automatically, the same way every time.
JUnit 5 + Mockito, 80% coverage	The tasting panel.	Nothing leaves the kitchen untasted. Taste fewer than 80% of the steps and the dish does not ship.

1.2 What the Résumé Line Means, Phrase by Phrase

Phrase	What it actually means in plain terms
Led the full lifecycle of multiple services	I owned more than one backend service from the blank page all the way to it running in production — not just picking up assigned tickets.
from architecture and design	I decided what each service should be responsible for, what its API looks like, how it stores data, and how it talks to the other services.
to development	I wrote the actual application code (Java, typically with Spring Boot).
testing	I wrote automated tests that re-check the code every single time anybody changes anything.
and maintenance	After release I kept it healthy: watched logs and metrics, fixed bugs, patched libraries, tuned slow queries, handled on-call issues.
shipping Docker images	Each service is packaged as a container — one self-contained bundle of the app plus everything it needs to run.
on Kubernetes (EKS)	Those containers run on Amazon's managed Kubernetes service, which schedules them, restarts them and scales them automatically.
with workload identities	Each service proves who it is to AWS using a short-lived, automatically-rotated identity instead of a permanent password stored in a config file.
and ingress gateways	Outside traffic comes in through one controlled HTTPS entry point that routes each web address to the correct service.
CI/CD in AWS CodePipeline	Build, test, package and deploy all happen automatically on every merge. Nobody releases by hand.
80%+ JUnit 5 / Mockito coverage	At least 80% of the code lines are exercised by automated unit tests, and the build refuses to release if that drops.

2. The Big Picture

Before going into detail, here is the whole journey on one page: from a developer typing code, through the automated pipeline, into the running cluster, and finally out to real users.

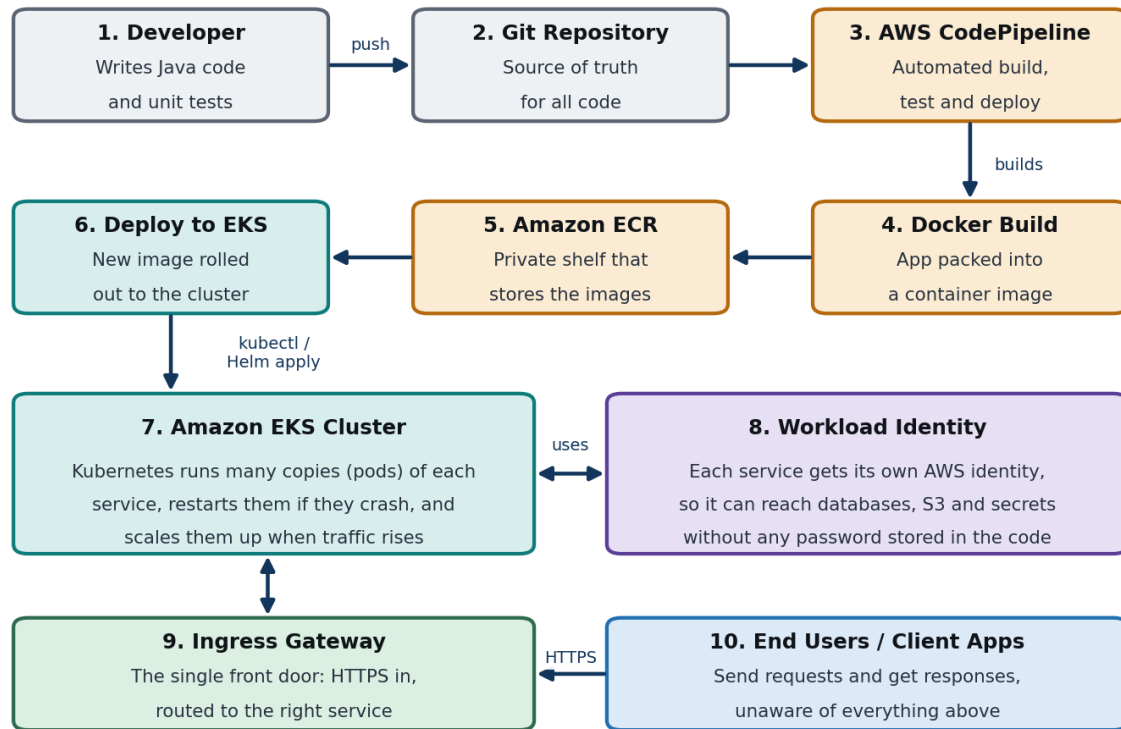


Figure 1 — End-to-end view: how code becomes a running service that users can reach.

Read Figure 1 in numbered order:

1. **Developer.** A person writes or changes the Java code for one service and writes unit tests alongside it.
2. **Git repository.** The change is pushed to the shared code repository, which is the single source of truth for what the service should be.
3. **AWS CodePipeline.** The moment the change lands, the pipeline wakes up on its own. No human presses a button.
4. **Docker build.** Once the code compiles and the tests pass, the application is packed into a container image — a sealed, self-contained bundle.
5. **Amazon ECR.** The finished image is pushed to a private registry: a shelf where every version of every image is stored and labelled.
6. **Deploy to EKS.** The pipeline tells the cluster to switch to the new image version.
7. **Amazon EKS cluster.** Kubernetes runs several copies of each service, restarts any copy that crashes, and adds more copies when traffic rises.

8. **Workload identity.** Each running copy is handed a temporary AWS identity so it can read the database, fetch a secret or write to storage — with no password in the code.
9. **Ingress gateway.** The single front door of the system. HTTPS traffic arrives here and is routed to whichever service owns that URL.
10. **End users.** People and client apps send requests and get responses, completely unaware of everything above.

The one idea behind the whole design

Every individual part is treated as disposable and replaceable, and the system as a whole is treated as permanent.

Any single container can be killed at any moment and the users should not notice, because another copy is already running and a replacement starts within seconds. That single principle is why containers, Kubernetes, health probes, rolling updates and automated pipelines all exist here.

3. Every Component Explained, End to End

Each component below is described the same way: what it is in plain terms, why it is needed, and what was actually done with it on this project.

3.1 The Microservices Themselves

What it is

A microservice is one small application that does one job and owns its own data. This project had several of them — for example an Order Service, a Payment Service and a User Service. Each has its own codebase, its own release schedule and its own database tables. They talk to each other over the network using HTTP APIs or messages.

The opposite approach is a monolith: one enormous program containing everything. A monolith is simpler at first, but every change forces a full redeployment, and one bad piece of code can take the entire product offline.

What is inside a typical service

Layer	Plain-terms job	Typical Java pieces
Controller / API layer	Receives incoming web requests, checks the input looks sane, and hands it inward.	@RestController, request and response objects
Service / business layer	Holds the actual rules: what a valid order is, when a payment can be retried.	@Service classes — this is where most unit tests aim
Repository / data layer	The only part that talks to the database. Reads and writes rows.	@Repository, Spring Data JPA interfaces
Model / DTO	The shapes of the data moving in, out and around.	Entities, DTOs, mappers
Configuration	Settings that change per environment: URLs, timeouts, feature flags.	application.yml, environment variables, ConfigMaps
Cross-cutting	Error handling, logging, security, retries, validation.	@ControllerAdvice, filters, interceptors

Why split things this way

- Independent releases. Fixing a payment bug does not require redeploying the user accounts service.
- Blast radius is contained. If the payment service falls over, people can still browse and log in.
- Targeted scaling. On a busy day only the service actually under load gets more copies, which is far cheaper than duplicating everything.
- Clear ownership. One service, one clear set of responsibilities, one team that knows it well.
- **The trade-off:** many more moving parts. That is exactly why containers, an orchestrator, an automated pipeline and strong monitoring are not optional extras here — they are what makes the split survivable.

3.2 “Full Lifecycle” — What Owning a Service Actually Involves

Leading the full lifecycle means being responsible for all five phases below, not just the coding one.

Phase	What happens in plain terms	What it produces
1. Architecture	Deciding the big shape: how many services, what each owns, how they communicate, where data lives, how failures are handled, how it scales.	Diagrams, decision records, a chosen approach and the reasons for it
2. Design	Deciding the details of one service: the exact API endpoints, request and response formats, database tables, error codes, timeouts and retries.	API contract, data model, sequence diagrams
3. Development	Writing the code, wiring the layers together, handling the awkward edge cases.	Working service, Dockerfile, Kubernetes manifests
4. Testing	Writing automated tests, checking behaviour, verifying the service still works alongside the others.	JUnit 5 test suites, mocks, a coverage report
5. Maintenance	Keeping it alive after launch: watching dashboards, fixing production bugs, upgrading dependencies, tuning performance, controlling cost.	Alerts, runbooks, patches, performance improvements

3.3 Docker Images — Packing the Application into a Sealed Box

The problem it solves

Software often behaves differently on a developer's laptop, on the test server and in production, because those machines have different Java versions, different libraries or different settings. This is the classic “but it works on my machine” problem. A container removes the difference by shipping the environment along with the application.

Image versus container — the distinction that trips people up

- **An image** is the sealed box sitting on a shelf: a read-only snapshot containing the compiled application, the Java runtime, the libraries and the start command. It does nothing until you run it.
- **A container** is a running copy of that image. One image can be started as 3, 30 or 300 identical containers — they are all the same recipe being cooked at once.

What goes into building one, step by step

1. **A Dockerfile is written.** It is a short recipe file: start from an official Java base image, copy in the built application, expose the port, and set the command that starts it.
2. **A multi-stage build is used.** The first stage has the full build tooling (Maven, the JDK, all source code) and compiles the application. The second stage keeps only the finished JAR and a slim Java runtime. The heavy build tools never reach production.
3. **The image is kept small and safe.** A slim base image, no build tools, no shell utilities that are not needed, and the app runs as a non-root user so a break-in cannot become an easy takeover.

4. **The image is tagged with the commit id.** Every image is labelled with the exact code version it was built from, for example `order-service:9f3c1ab`. Tagging everything “latest” is avoided, because then nobody can tell which version is actually running or roll back cleanly.
5. **The image is immutable.** Once built it is never edited. A change means building a brand-new image with a new tag. This is what makes a rollback simply “go back to the previous tag”.

3.4 Amazon ECR — The Private Shelf for Images

ECR (Elastic Container Registry) is the private storage where built images live. When the cluster needs to start a service, it pulls the image from here.

- **Private by default.** Only the pipeline can push, and only the cluster's nodes can pull. The application is not published to the public internet.
- **Every version is kept.** Each tag is a complete historical snapshot, which is what makes rolling back to last week's version a matter of seconds.
- **Vulnerability scanning.** ECR can scan each pushed image for known security holes in its libraries and flag them before they reach production.
- **Lifecycle rules.** Old, unused images are deleted automatically so storage costs do not grow forever.

3.5 Kubernetes — The Manager That Keeps Everything Running

Containers on their own are just boxes. Something has to decide which machine each box runs on, notice when one dies, start a replacement, share traffic between the copies, and add more copies when things get busy. That something is Kubernetes.

Kubernetes works on a simple promise: you describe the state you want in a text file, and it continuously works to make reality match that description. You never say “start a container on server 4.” You say “there should always be three healthy copies of this service,” and Kubernetes takes it from there, forever.

The building blocks, in plain terms

Object	What it is, in plain terms
Pod	The smallest unit Kubernetes runs — in practice, one running container of your service. It gets its own internal IP address and is treated as disposable: pods are created and destroyed constantly.
ReplicaSet	The counter that makes sure the requested number of pods exists. If one dies, it immediately starts another.
Deployment	The instruction sheet you actually write: which image to run, how many copies, how much memory, and how to roll out a new version safely. It manages the ReplicaSet for you and remembers previous versions so it can roll back.
Service	A stable internal address and name in front of pods that keep changing. Pods come and go with different IPs; the Service name stays the same and shares traffic across whichever pods are currently healthy.

Object	What it is, in plain terms
Ingress	The rulebook for outside traffic: which web address and URL path goes to which Service.
ConfigMap	Ordinary settings kept outside the image — URLs, timeouts, feature switches — so the same image can run in test and in production.
Secret	The same idea for sensitive values, stored separately and handled more carefully.
Namespace	Folders inside the cluster used to keep environments or teams apart, for example dev, staging and prod.
ServiceAccount	The identity a pod carries inside the cluster. This is the hook that workload identity attaches to (section 3.7).
Liveness probe	A repeated “are you still alive?” check. If a pod stops answering, Kubernetes kills and restarts it automatically.
Readiness probe	A “are you ready for traffic yet?” check. A pod that is still warming up receives no requests until it says yes — this is what makes restarts invisible to users.
Resource requests and limits	The guaranteed amount of CPU and memory a pod gets, and the ceiling it may never exceed. Stops one misbehaving service from starving the others.
Horizontal Pod Autoscaler	Watches load and adds or removes pod copies automatically, then scales back down when it quietens.

3.6 Amazon EKS — Kubernetes Without Running Kubernetes Yourself

Kubernetes has two halves. EKS is Amazon running the difficult half for you.

- **The control plane (Amazon's responsibility).** The brain that makes all the decisions and stores the cluster's state. Running it yourself means keeping several servers highly available, backed up, patched and upgraded. With EKS, Amazon does that.
- **The worker nodes (your responsibility).** The actual machines where your pods run. These are EC2 servers grouped into node groups inside your own private network, and they can scale in and out as demand changes.

The practical benefit: the team spends its time on the services and how they behave, not on maintaining cluster infrastructure. EKS also plugs straight into the rest of AWS — load balancers, IAM permissions, networking and CloudWatch monitoring all work together without extra glue.

3.7 Workload Identities — Access Without Passwords

The problem

A service needs to read a database, fetch a file from storage or pull a secret. The old way was to put an access key and a secret key into a config file or an environment variable. That is dangerous: the key is long-lived, it

gets copied into images and laptops, it leaks into logs and screenshots, it is shared between services so nobody can tell who did what, and rotating it is a painful manual job everybody puts off.

The solution in one sentence

Instead of a permanent password, each pod is handed a short-lived, automatically-renewed identity token that AWS trusts, and that token can be exchanged for temporary credentials tied to one narrowly-scoped IAM role.

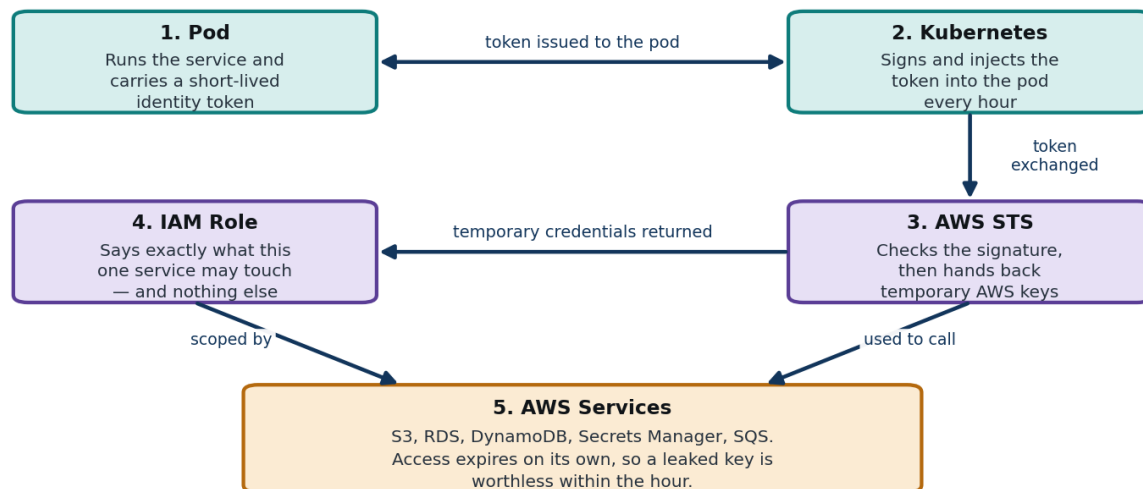


Figure 2 — How a running pod obtains AWS access without any stored password.

Step by step, this is what happens every time:

1. The service is deployed with a Kubernetes ServiceAccount — its name badge inside the cluster.
2. That ServiceAccount is linked to one specific AWS IAM role, and the role's trust policy says: “only this ServiceAccount, in this cluster, in this namespace, may assume me.”
3. When the pod starts, Kubernetes automatically places a signed, time-limited token inside it. Nobody types it, and it is never stored in the image or the code.
4. When the application makes an AWS call, the AWS SDK inside the application picks up that token and sends it to AWS STS (the Security Token Service).
5. STS verifies the signature against the cluster's trusted identity provider and confirms the ServiceAccount is allowed to assume that role.
6. STS returns temporary credentials that expire after roughly an hour. The SDK refreshes them silently before they run out.
7. The service uses those credentials to call only what its IAM role permits — nothing more.

Why this matters

No secret to leak. There is no long-lived key sitting in a file, an image or a laptop.

Least privilege, per service. The order service can be allowed to read the orders bucket and nothing else. A mistake in one service cannot touch another's data.

Automatic rotation. Credentials expire on their own, so a leaked one is worthless within the hour.

Clean auditing. AWS logs show which service made which call, because each service has its own identity rather than sharing one.

3.8 Ingress Gateways — The Single Guarded Front Door

The services run inside a private network with no direct route from the internet. Something has to accept public traffic and pass it inward safely. That is the ingress gateway.

Three words get used loosely here, and they are three different things:

- **The Ingress resource** is the rulebook — a text file listing which hostname and URL path maps to which service.
- **The Ingress controller** is the program running in the cluster that reads that rulebook and makes it real.
- **The load balancer** is the actual AWS Application Load Balancer with a public address that traffic physically arrives at.

What it does for you

Job	What it means in plain terms
One entry point	One public address and one certificate for the whole system, instead of exposing every service separately to the internet.
HTTPS termination	Encryption is unwrapped here, so each individual service does not have to manage certificates.
Routing by host and path	/orders goes to the order service, /payments to the payment service, /users to the user service — all under one domain.
Health-aware traffic	Requests are only sent to pods that have reported themselves ready, which is why a deployment can happen mid-day without downtime.
Central protection	Rate limiting, request size limits, timeouts and a web application firewall are configured once, in one place, for everything behind it.
A stable public surface	Pods restart and move constantly behind the scenes. The address the outside world uses never changes.

3.9 CI/CD in AWS CodePipeline — The Automated Assembly Line

CI/CD stands for Continuous Integration and Continuous Delivery. In plain terms: every time somebody changes the code, a machine — not a person — rebuilds it, tests it, packages it and releases it, following exactly the same steps every single time.

Manual releases fail for predictable reasons: a step gets forgotten, a build is made from the wrong branch, the person who knows the procedure is on holiday, and every release is slightly different from the last. Automation removes all of that.

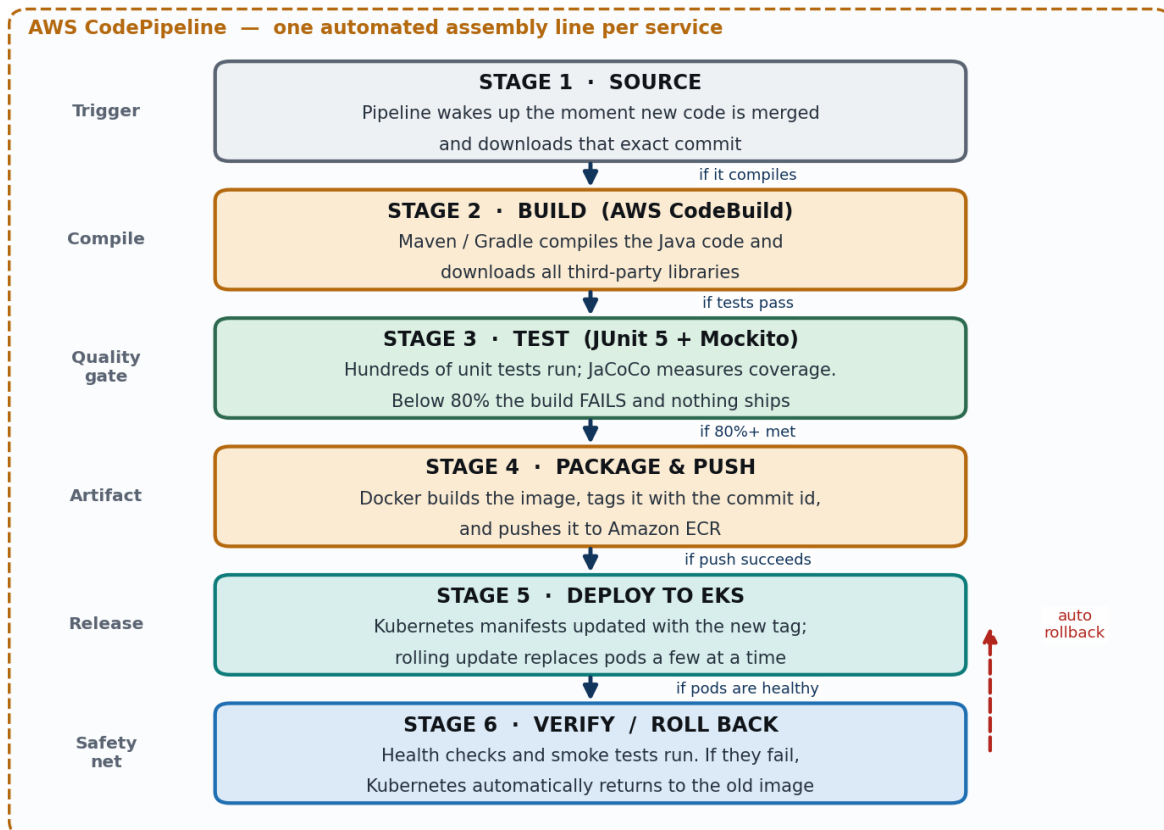


Figure 3 — The six pipeline stages. Each stage must pass before the next begins.

The pieces of the pipeline

- **CodePipeline** is the conductor. It defines the ordered stages and moves the work from one to the next, stopping the moment anything fails.
- **CodeBuild** is the worker that actually does the compiling, testing and image building, following a buildspec file kept alongside the code.
- **Artifacts** — the outputs of each stage, such as the compiled JAR and the test reports — are passed along and stored, so any release can be inspected later.

- **Environment promotion.** The same image travels from dev to staging to production. Because it is literally the same image, what you tested is what you released — not a rebuild that might differ.
- **Approval gates.** Production releases can require a human tick, while lower environments flow through untouched.

Stage	What happens	What stops it
1. Source	The pipeline detects a merge and downloads that exact commit.	Nothing — this is the trigger.
2. Build	Maven or Gradle compiles the code and resolves every library.	A compile error, or a missing or vulnerable dependency.
3. Test	All JUnit 5 tests run; JaCoCo measures how much code they touched.	Any failing test, or coverage below the 80% threshold.
4. Package & push	Docker builds the image, tags it with the commit id, pushes it to ECR.	A failed build, or a blocking image vulnerability scan.
5. Deploy	The Kubernetes deployment is updated to the new image tag; a rolling update begins.	The cluster rejecting an invalid manifest.
6. Verify	Health checks and smoke tests confirm the new pods are genuinely working.	Unhealthy pods — which triggers an automatic rollback.

3.10 Testing — JUnit 5, Mockito and 80%+ Coverage

What a unit test is

A unit test is a small piece of code whose only purpose is to check that another small piece of code does what it is supposed to. It runs in a fraction of a second, needs no database and no network, and it either passes or fails with no human judgement involved. A service typically has hundreds of them, and they all run on every single change.

JUnit 5 — the framework that runs the tests

- **@Test** marks a method as a test so the framework knows to run it.
- **@BeforeEach** and **@AfterEach** set up fresh conditions before each test and clean up afterwards, so tests never contaminate one another.
- **Assertions** are the actual checks: this result should equal that value, this call should throw that specific error.
- **@ParameterizedTest** runs the same test against many different inputs — valid, empty, negative, enormous — without copying and pasting the test.
- **@DisplayName** and **@Nested** keep the suite readable, so a failure report reads like a sentence describing what broke.

Mockito — the stunt doubles

Most code depends on other things: a database, another service, a payment provider. Calling those for real in a unit test would be slow, unreliable and occasionally expensive. Mockito replaces each dependency with a stand-in that you control completely — a stunt double instead of the real actor.

- **Stubbing.** You declare in advance what the stand-in returns, so you can test how your code reacts to a specific answer.
- **Verification.** You check afterwards that your code called the dependency the right number of times with the right values — for instance, that a failed payment was never charged twice.
- **Failure simulation.** You can make the stand-in throw a timeout or a database error on demand. This is how error handling gets tested at all, since real failures are hard to arrange deliberately.
- **@Mock and @InjectMocks** create the stand-ins and slot them into the class under test automatically.

What “80%+ coverage” means and why it is enforced

Coverage is the percentage of your code lines that were actually executed while the tests ran. If your service has 10,000 lines and the tests exercised 8,200 of them, that is 82% coverage. A tool called JaCoCo measures this during the build and produces a report.

The 80% figure is not a decoration — it is a gate wired into the pipeline. If a change drops coverage below the threshold, the build fails and nothing is released. That forces tests to be written alongside features rather than promised for later.

An honest caveat worth knowing

Coverage measures how much code was run, not whether the checks were meaningful. A test that executes a method but asserts nothing still counts toward the percentage.

So the number is a floor, not a goal: it guarantees no large area of the codebase is completely untested, while code review remains responsible for whether the tests actually check the right behaviour.

3.11 Observability and Maintenance — Life After the Release

Shipping is the start, not the finish. Maintenance is the part of the lifecycle that runs forever.

Activity	What it involves in plain terms
Logs	Every service writes a running diary of what it did. Those diaries are collected centrally so you can trace one request across several services instead of logging into machines.
Metrics	Numbers sampled continuously: requests per second, error rate, response time, CPU and memory. These show trends that a single log line never would.
Alarms	Thresholds that page a human automatically — for example, error rate above 2% for five minutes — so problems are found before customers report them.
Dashboards	One screen showing the health of all services at a glance, used daily and during incidents.
Tracing	Following a single user request as it hops between services, which is how you find which hop is the slow one.

Activity	What it involves in plain terms
Patching and upgrades	Keeping Java, Spring, libraries, base images and Kubernetes versions current, mostly to close security holes.
Performance tuning	Finding slow database queries, adding caching, right-sizing CPU and memory requests.
Cost control	Making sure autoscaling scales back down, old images are cleaned up, and nothing over-provisioned is quietly running up a bill.

4. The Workflows, End to End

Sections 2 and 3 covered the parts. This section covers the motion — what actually happens, in order, in the four situations that matter.

4.1 Workflow A — From a Developer's Keystroke to Live in Production

This is the build-and-release journey, and it is fully automatic after step 3.

1. **Write the change.** A developer implements a feature or fix on a branch, and writes the unit tests for it at the same time.
2. **Run tests locally.** The tests are run on the laptop first, so obvious breakage never reaches the shared pipeline.
3. **Open a pull request.** Another engineer reviews the change: is the logic right, are the tests meaningful, does it fit the architecture?
4. **Merge.** Once approved, the change lands in the main branch. This is the last manual action in the whole journey.
5. **Pipeline triggers.** CodePipeline detects the merge within seconds and pulls that exact commit.
6. **Compile.** CodeBuild fetches every library and compiles the Java code. A compile failure stops everything here.
7. **Run the test suite.** Hundreds of JUnit 5 tests execute, with Mockito standing in for databases and external services. One failure stops the pipeline.
8. **Check coverage.** JaCoCo produces the coverage report. Below 80%, the build is failed deliberately, even though every test passed.
9. **Build the image.** Docker packages the compiled application into an image using the multi-stage Dockerfile.
10. **Tag and scan.** The image is tagged with the commit id and scanned for known vulnerabilities in its libraries.
11. **Push to ECR.** The image is uploaded to the private registry, where it now sits permanently alongside every previous version.
12. **Update the deployment.** The pipeline updates the Kubernetes deployment to point at the new image tag — the moment the release truly begins.
13. **Rolling update.** Kubernetes starts one new pod, waits for its readiness probe to pass, sends it live traffic, then retires one old pod — and repeats. At no point are there zero healthy pods, so users see no downtime.
14. **Verify.** Health checks and smoke tests confirm the new version genuinely works, not merely that it started.
15. **Roll back if needed.** If the new pods fail their checks, Kubernetes stops the rollout and returns to the previous image automatically. The old version was never deleted, so this takes seconds.

16. **Monitor.** Dashboards and alarms are watched after release, since some problems only appear under real traffic.

4.2 Workflow B — What Happens to One Single User Request

A user taps a button. Here is every hop that follows, in order.

1. **DNS lookup.** The client asks Route 53 what address belongs to the domain name and gets back the load balancer's address.
2. **Secure connection.** An HTTPS connection is established. The certificate is presented and the traffic is encrypted from the very first byte.
3. **Edge protection.** The request passes the web application firewall and rate limiting, which drop obviously malicious or abusive traffic before it costs anything.
4. **Load balancer.** The Application Load Balancer receives the request in the public part of the network and forwards it into the cluster.
5. **Ingress routing.** The ingress controller reads the hostname and URL path and decides this request belongs to, say, the order service.
6. **Kubernetes Service.** The request reaches the order service's stable internal address, which picks one of the currently healthy pods.
7. **Pod receives it.** A specific running container gets the request. Which one is chosen does not matter, because they are identical.
8. **Controller layer.** Inside the application, the controller validates the incoming data and rejects anything malformed straight away.
9. **Business logic.** The service layer applies the actual rules — stock checks, pricing, permissions.
10. **Credentials on demand.** If AWS resources are needed, the SDK uses the pod's workload identity to obtain temporary credentials, silently and without any stored password.
11. **Data access.** The repository layer reads or writes the database, or fetches a file from storage.
12. **Downstream calls.** If another service is needed, it is called through its internal Service name, with a timeout and a retry policy so one slow dependency cannot hang the whole request.
13. **Response returns.** The answer travels back out the same path: pod, Service, ingress, load balancer, user.
14. **Evidence is recorded.** Throughout, the request is logged and its timing and outcome are recorded as metrics, which is what makes later debugging possible at all.

4.3 Workflow C — How a Pod Gets Permission to Use AWS

This is Figure 2 restated as a sequence. It repeats quietly, roughly every hour, for the life of every pod.

1. The pod starts, carrying a ServiceAccount that has been linked to one specific IAM role.
2. Kubernetes writes a signed, short-lived identity token into the pod's filesystem.

3. The application makes an AWS call. The SDK notices the token and calls AWS STS with it.
4. STS checks the signature against the cluster's registered identity provider, and checks that this ServiceAccount is permitted to assume that role.
5. STS returns temporary credentials scoped strictly to that role's permissions.
6. The call proceeds — reading the database, fetching a secret, writing to storage — and is recorded in AWS's audit log under that service's own identity.
7. Before the credentials expire, the SDK quietly fetches new ones. Nobody rotates anything by hand, ever.

4.4 Workflow D — What Happens When Things Go Wrong

The design is judged on failure days, not good days. These are the common failures and the automatic response to each.

What goes wrong	What happens automatically	What a human does
A pod crashes	The liveness probe fails, Kubernetes kills the pod and starts a fresh one. Traffic goes to the other copies meanwhile, so users notice nothing.	Read the logs afterwards to find why it crashed and fix the root cause.
A whole server dies	Kubernetes reschedules its pods onto healthy nodes; the node group replaces the lost machine.	Usually nothing during the event — review capacity afterwards.
Traffic suddenly spikes	The autoscaler adds pod copies, and more nodes if needed. The ingress spreads load across all of them.	Check the scaling limits and cost impact.
A bad version is released	Health checks fail, the rolling update halts, and Kubernetes returns to the previous image.	Investigate using the failed build's artifacts, then fix forward.
A dependency is slow	Timeouts and retries stop the request from hanging indefinitely and prevent the failure spreading upstream.	Trace the slow hop and address it at the source.
A library vulnerability appears	Image scanning flags it and can block the release.	Upgrade the library; the pipeline rebuilds, retests and redeploys automatically.
A secret needs rotating	Credentials expire on their own; new ones are issued silently. Nothing needs redeploying.	Nothing — this is precisely the manual job workload identity removed.

5. The Architecture Diagram, Explained Layer by Layer

Figure 4 shows the running system — not how code gets built, but what exists and talks to what once everything is live. Read it strictly top to bottom: that is the direction a user request travels.

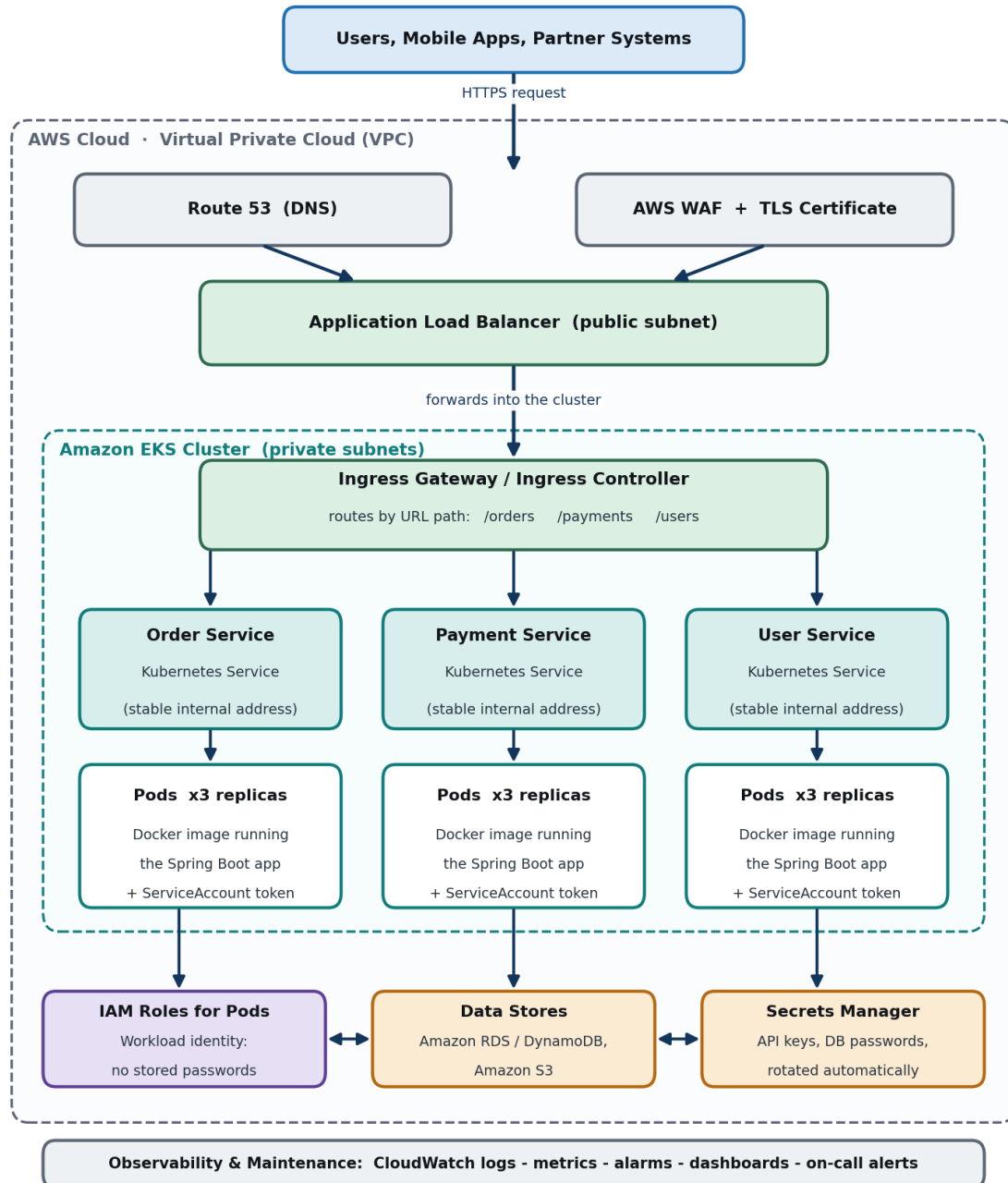


Figure 4 — Runtime architecture: from the public internet down to data storage.

5.1 Every Element in the Diagram

Element	What it is and why it sits exactly there
Users, mobile apps, partner systems	Everything outside your control that sends requests. They only ever know one public address and never see anything below the load balancer.
VPC (Virtual Private Cloud)	The outer dashed boundary: your own private slice of the AWS network. Nothing inside is reachable from the internet unless deliberately exposed.
Route 53 (DNS)	The phone book. Translates the domain name into the load balancer's address. First stop for every request.
WAF and TLS certificate	The security layer at the edge. The firewall blocks common attack patterns and abusive traffic; the certificate is what makes the connection HTTPS.
Application Load Balancer	The only component with a public address, and the only one in the public subnet. It receives all external traffic and forwards it inward.
EKS cluster (inner dashed box)	Everything inside sits in private subnets with no direct route from the internet. This is the boundary that makes the services unreachable except through the front door.
Ingress gateway / controller	The router inside the cluster. It reads the URL path and sends the request to the right service. One entry point, many destinations.
Kubernetes Services	One stable internal name per microservice. Pods behind it are constantly replaced; the name never changes, which is what lets everything else stay stable.
Pods (three replicas each)	The actual running copies of the Docker image. Three copies means one can die or be replaced during a deployment while the other two keep serving. Each carries its ServiceAccount token.
IAM roles for pods	The permission definitions behind workload identity. Each service maps to its own role, so each can reach only what it genuinely needs.
Data stores	Managed databases and object storage, kept outside the cluster on purpose: pods are disposable, data must never be. Backups, replicas and patching are AWS's job.
Secrets Manager	Where sensitive values live. Services fetch them at runtime using their workload identity, so secrets never appear in code, images or manifests.
Observability strip	Logs, metrics, alarms and dashboards collected from every layer above. Drawn across the bottom because it touches all of them, not just one.

5.2 The Five Boundaries Worth Noticing

1. **Public versus private.** Only the load balancer is publicly addressable. Every service and every database sits in a private subnet. An attacker cannot reach a pod directly, only through the front door and its rules.
2. **Stateless versus stateful.** Pods hold no permanent data, which is what makes them safe to destroy at any moment. All state lives in the managed databases and storage below them.

3. **Identity per service, not per cluster.** Permissions are drawn at the individual service level. A flaw in one service does not become access to everything.
4. **One entry point, many services.** Adding a fourth or fifth microservice means adding a routing rule, not a new public address, a new certificate or a new firewall configuration.
5. **Observability spans everything.** Monitoring is deliberately drawn across all layers, because in a distributed system the hardest question is not “is it broken?” but “which of the seven hops is broken?”

6. Glossary

Term	Plain-terms meaning
Microservice	One small application that does one job and owns its own data.
Monolith	The opposite: one large application that does everything.
Container	A running application packaged with everything it needs, isolated from its host.
Docker image	The read-only snapshot a container is started from.
Dockerfile	The recipe file describing how to build an image.
Registry / ECR	Private storage where built images are kept and versioned.
Kubernetes	The system that runs, restarts, scales and connects containers automatically.
EKS	Amazon's managed Kubernetes: AWS runs the control plane for you.
Pod	The smallest thing Kubernetes runs — in practice one container of your service.
Deployment	The instruction sheet describing which image to run, how many copies, and how to update safely.
Service (Kubernetes)	A stable internal address in front of pods that keep changing.
Ingress	The rules that decide which outside traffic reaches which service.
Ingress controller	The program inside the cluster that enforces those rules.
Load balancer	Spreads incoming traffic across several healthy copies.
Replica	One of several identical copies of a service running at once.
Rolling update	Replacing old pods with new ones a few at a time, so there is no downtime.
Rollback	Returning to the previous known-good image version.
Liveness probe	A repeated check asking “is this pod still alive?”
Readiness probe	A repeated check asking “is this pod ready for traffic yet?”
Autoscaling	Adding or removing copies automatically as load changes.
Namespace	A folder inside the cluster used to separate environments or teams.
ServiceAccount	A pod's identity inside the cluster.
Workload identity	Giving a pod short-lived AWS credentials instead of a stored password.
IAM role	A named set of AWS permissions that something can temporarily assume.
STS	The AWS service that exchanges a trusted token for temporary credentials.
CI/CD	Automatically building, testing and releasing on every code change.

Term	Plain-terms meaning
CodePipeline	The AWS service that orchestrates those automated stages.
CodeBuild	The AWS service that performs the compiling, testing and image building.
Artifact	A file produced by a build stage, such as a compiled JAR or a test report.
Unit test	A fast automated check that one small piece of code behaves correctly.
JUnit 5	The Java framework that defines and runs those tests.
Mockito	A library that replaces real dependencies with controllable stand-ins during tests.
Mock	A fake stand-in for a real dependency, used to keep tests fast and predictable.
Code coverage	The percentage of code lines actually executed while the tests ran.
JaCoCo	The tool that measures Java code coverage during the build.
VPC	Your own private network inside AWS.
Subnet	A subdivision of that network; public ones can reach the internet, private ones cannot.

7. How to Explain This Out Loud

7.1 The 30-Second Version

Say this

“I owned several backend microservices end to end — designing them, building them in Java, testing them, and keeping them running. Each one ships as a Docker container onto Amazon's managed Kubernetes, behind a single ingress gateway that handles HTTPS and routing. Every service authenticates to AWS with its own short-lived workload identity, so there are no stored credentials anywhere. The whole build-test-release path is automated in AWS CodePipeline, with a hard gate at 80% unit-test coverage — if coverage drops, the build fails and nothing ships.”

7.2 The Two-Minute Version

Walk through it in this order, because it follows the natural flow of the work:

1. **The problem.** A single large application meant every change risked everything and releases were slow and manual.
2. **The shape.** Split into focused services, each owning its own data and its own release cycle.
3. **The packaging.** Docker so a service behaves identically everywhere, tagged by commit so any version can be reproduced or rolled back exactly.
4. **The platform.** EKS, so the cluster self-heals, scales with demand and deploys without downtime.
5. **The front door.** One ingress gateway handling TLS, routing and edge protection, so no service is exposed directly.
6. **The security posture.** Workload identity per service: temporary, automatically-rotated credentials scoped to least privilege, with no secrets in code or images.
7. **The automation.** CodePipeline builds, tests, scans, packages and deploys on every merge, with automatic rollback on failure.
8. **The quality bar.** JUnit 5 and Mockito with an enforced 80% coverage gate, so quality is a build condition rather than a good intention.
9. **The result.** Faster, safer releases; failures contained to one service; no manual credential handling; and problems visible on a dashboard before customers report them.

7.3 Questions You Should Expect

Likely question	The short honest answer
Why microservices rather than a monolith?	Independent releases, contained failures, and the ability to scale only the part under load. The cost is operational complexity, which is why the automation and monitoring exist.
What does workload identity actually replace?	Long-lived AWS access keys sitting in config files or environment variables. It removes the secret entirely rather than protecting it better.

Likely question	The short honest answer
Why 80% and not 100%?	80% is a realistic floor that guarantees no large area is untested, without pushing the team into writing meaningless tests just to reach a number.
Is high coverage the same as good tests?	No. Coverage proves code was executed, not that the assertions were meaningful. It is enforced as a floor; code review handles test quality.
How do you deploy without downtime?	A rolling update plus readiness probes: new pods only receive traffic once they report ready, and old pods are only retired after that.
What happens if a release is bad?	Health checks fail, the rollout halts, and Kubernetes returns to the previous image automatically. The old image is still in ECR, so it takes seconds.
What was the hardest part?	Usually the distributed-systems reality: timeouts, retries, partial failures and tracing one request across several services.